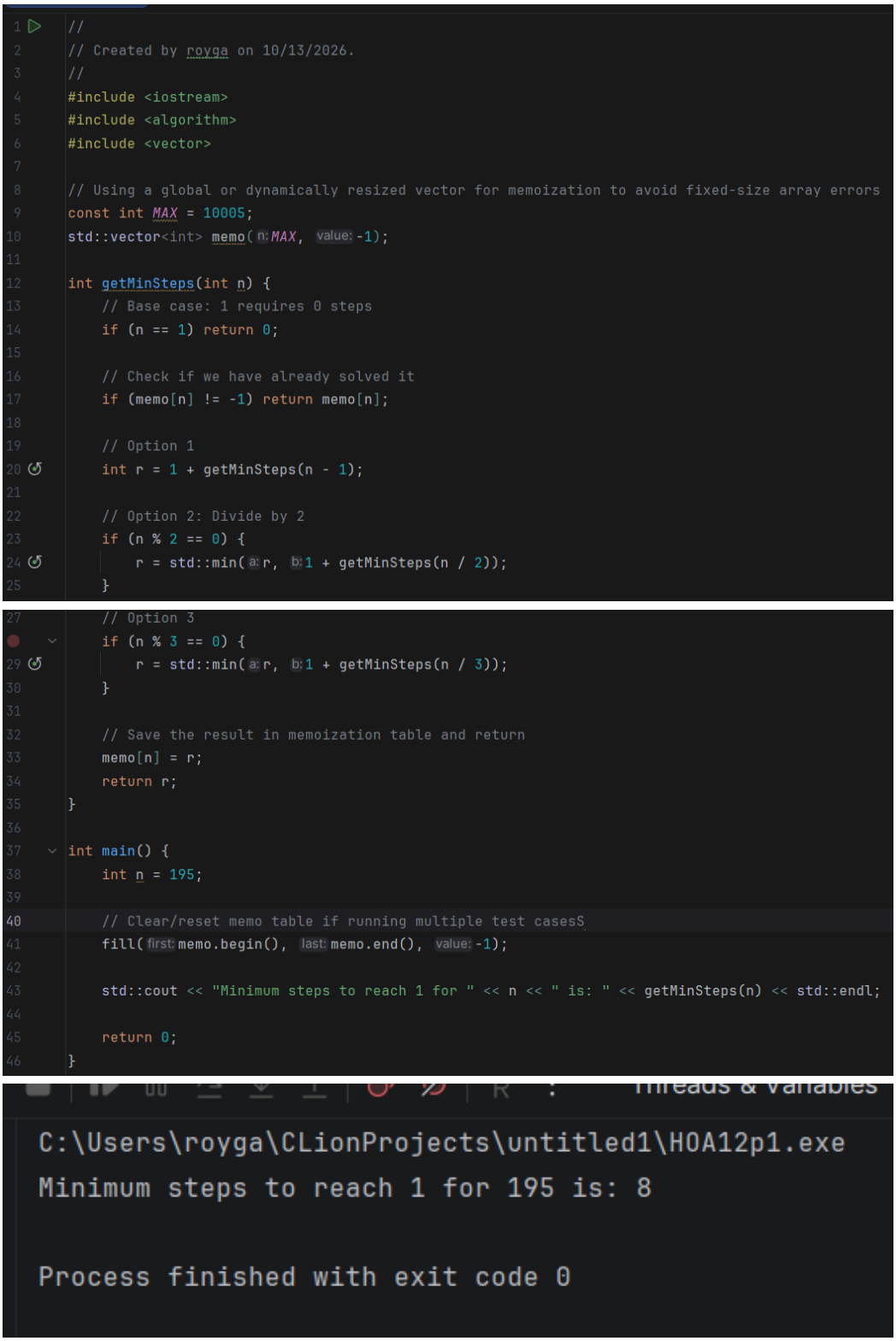
COURSE CODE/TITLE: CPE 010-CPE21S1 - Data Structures and Algorithms	**SECTION: CPE21S1**
DATE PERFORMED: 13/10/2026	**DATE SUBMITTED: 15/10/2026**
NAME: Mendoza, Roy Gabriel I. **Pilapil, Loreen Jae D.**	**INSTRUCTOR: Engr. Jimlord Quejado**

ACTIVITY NO. 12p1 Algorithmic Strategies

1. PROCEDURE OUTPUT

Strategy	Algorithm	Analysis
Recursion	**Quick Sort**	Quick sort uses the recursion strategy because after selecting a pivot and parting the array into left and right sub arrays, the algo repeatedly calls itself on these sub arrays until the element is reached or ended.
Brute Force	**Bubble Sort**	Bubble sort is a brute force strategy because we compare and swap “each” adjacent pairs and it does not matter if the pairs are already sorted and properly placed, this happens until the end of the array.
Backtracking	**Tree Traversal**	Tree traversal uses the backtracking strategy because in the three cases that we have, most of the algorithm of the tree traversal went from the top to bottom then if there are more children as we know, we backtrack to the parent and go back to the sibling.
Greedy	**Selection Sort**	Selection sort is considered as a greedy approach because at every iteration, it chooses the lowest value in the unsorted region and puts it into its proper place without consideration to its choice.
Divide-and-Conquer	**Merge Sort**	Merge sort is one of the most notable divide and conquer strategies that we have because this algorithm divides the array into 2 until there are only 2 elements left for each sub array to be compared, and then we identify if the adjacent elements are sorted already or needed to be sorted still.

Table 12-1. Algorithmic Strategies and Example

Screenshot	
Analysis	

```cpp
//
// Created by royga on 10/13/2026.
//
#include <iostream>
#include <algorithm>
#include <vector>

// Using a global or dynamically resized vector for memoization to avoid fixed-size array errors
const int MAX = 10005;
std::vector<int> memo(n:MAX, value:-1);

int getMinSteps(int n) {
    // Base case: 1 requires 0 steps
    if (n == 1) return 0;

    // Check if we have already solved it
    if (memo[n] != -1) return memo[n];

    // Option 1
    int r = 1 + getMinSteps(n - 1);

    // Option 2: Divide by 2
    if (n % 2 == 0) {
        r = std::min(a:r, b:1 + getMinSteps(n / 2));
    }
    // Option 3
    if (n % 3 == 0) {
        r = std::min(a:r, b:1 + getMinSteps(n / 3));
    }

    // Save the result in memoization table and return
    memo[n] = r;
    return r;
}

int main() {
    int n = 195;

    // Clear/reset memo table if running multiple test casesS
    fill(first:memo.begin(), last:memo.end(), value:-1);

    std::cout << "Minimum steps to reach 1 for " << n << " is: " << getMinSteps(n) << std::endl;

    return 0;
}
```

```
C:\Users\royga\CLionProjects\untitled1\HOA12p1.exe
Minimum steps to reach 1 for 195 is: 8

Process finished with exit code 0
```

MENDOZA	This program or memoization approach solves the problem recursively by storing the pre-computed solutions, then if there is a new problem, checks if the new problem is already solved in the past solutions and if it is not, then solve it again by using the three methods that we have.
PILAPIL	This program determines the minimum number of steps required for an integer to reach 1 using Top-down recursion and Memoization. For example, using the input 195: it first divides by 3 to get 65, subtracts 1 to get 64, and then divides by 2 repeatedly through 32, 16, 8, 4, and 2, before subtracting 1 to reach 1.

Table 12-2. Memoization Implementation

Screenshot

```cpp
#include <iostream>
#include <vector>
#include <climits>

int getMinSteps(int n) {
if (n <= 1) return 0;

std::vector<int> dp(n + 1);
dp[1] = 0; // base case

for (int i = 2; i <= n; i++) {
int res = dp[i - 1]; // option 1: subtract 1

if (i % 2 == 0) {
res = std::min(res, dp[i / 2]); // option 2: divide by 2
}
if (i % 3 == 0) {
res = std::min(res, dp[i / 3]); // option 3: divide by 3
}

dp[i] = 1 + res; // add 1 step for the chosen operation
}

return dp[n];
}

int main() {
int n = 195;
std::cout << "Minimum steps to reach 1 for " << n << " is: " << getMinSteps(n) << std::endl;
return 0;
}
```

```
Minimum steps to reach 1 for 195 is: 8
PS C:\Users\TIPQC\OneDrive\Documents\CPE21S1\HOA12P1>
```

Analysis

MENDOZA	This program uses the bottom up dp approach by building the solutions from the base of zero up to the N that we have set it to. Unlike the first one where we already have the max as the template and where we get the steps from, this one fills the solution sequentially.

PILAPIL	This program determines the minimum number of steps required for an integer to reach 1 using Bottom-up. It works by starting at 1 then building up to n.

Table 12-3. Bottom-Up Dynamic Programming Implementation

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

ILO B: Create C++ code to implement various algorithmic strategies & ILO C: Create C++ code to solve problems using learned algorithmic strategies

CODE

```cpp
#include <iostream>
#include <vector>
#include <string>

int countPaths(const std::vector<std::vector<int>>& mat, int i, int j, int cost) {
    // base case
    if (i < 0 || j < 0 || cost < 0) return 0;


    // base case
    if (i == 0 && j == 0) return (mat[0][0] == cost) ? 1 : 0;


    // recursive case
    return countPaths(mat, i - 1, j, cost - mat[i][j]) +
           countPaths(mat, i, j - 1, cost - mat[i][j]);
}


// Prints paths
void printPaths(const std::vector<std::vector<int>>& mat, int i, int j,
                int sum, int cost, std::string moves) {
    sum += mat[i][j];
    if (sum > cost) return;
    if (i == (int)mat.size() - 1 && j == (int)mat[0].size() - 1) {
        if (sum == cost) std::cout << "  " << moves << "\n";
        return;
    }
    if (j + 1 < (int)mat[0].size()) printPaths(mat, i, j + 1, sum, cost, moves + "R");
    if (i + 1 < (int)mat.size())    printPaths(mat, i + 1, j, sum, cost, moves + "D");
}
```

```cpp
int main() {
    std::vector<std::vector<int>> mat = {
        {4, 7, 1, 6},
        {5, 7, 3, 9},
        {3, 2, 1, 2},
        {7, 1, 6, 3}};
    int cost = 25;
    int M = mat.size(), N = mat[0].size();


    std::cout << "Matrix:\n";
    for (const auto& row : mat) {
        for (int v : row) std::cout << v << " ";
        std::cout << "\n";
    }

    std::cout << "\nTarget cost    : " << cost << "\n";
    std::cout << "Number of paths: " << countPaths(mat, M - 1, N - 1, cost) << "\n";
    std::cout << "Paths found:\n";
    printPaths(mat, 0, 0, 0, cost, "");
    return 0;
}
```

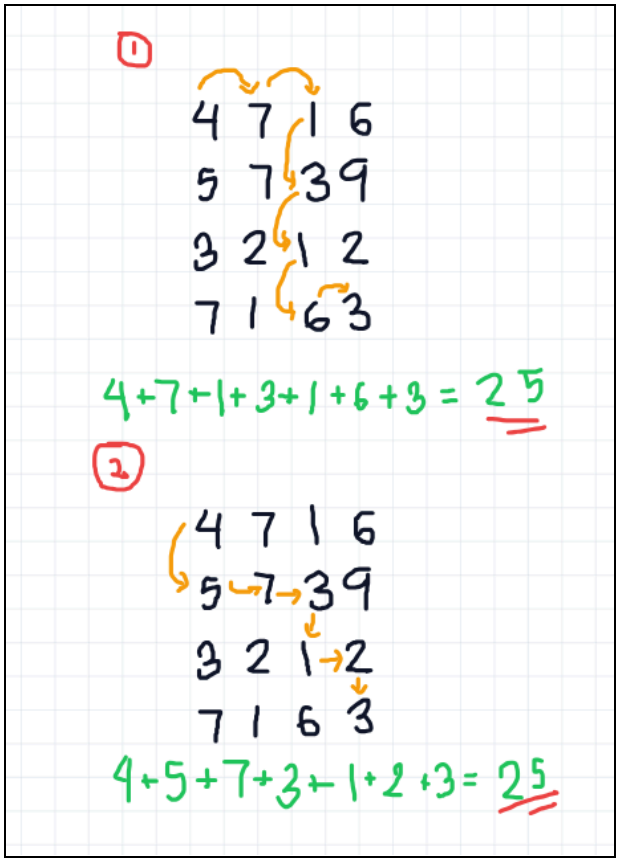
```
Matrix:
4 7 1 6
5 7 3 9
3 2 1 2
7 1 6 3

Target cost    : 25
Number of paths: 2
Paths found:
  RRDDDR
  DRRDRD
```

PSEUDOCODE

```
countPaths(mat, i, j, cost):
   if cost < 0 or i < 0 or j < 0:   return 0
   if i == 0 and j == 0:          return 1 if mat[0][0] == cost else 0
   return countPaths(mat, i-1, j, cost - mat[i][j])
       + countPaths(mat, i, j-1, cost - mat[i][j])

answer = countPaths(mat, M-1, N-1, targetCost)
```



ANALYSIS

MENDOZA	I saw that the program counts how many paths go from the left cell to the bottom-right cell of a 4x4 matrix. The program insists that the total cost of each path must be 25 and movements are only allowed to go down. The program successfully finds two paths and both paths have a sum of 25. The program also uses recursion in the countPaths function, which calls itself to solve parts of the problem.
PILAPIL	The program counts how many paths go from the top-left cell to the bottom-right cell of a 4 × 4 matrix. The total cost of each path must be 25, and are only allowed to move right or down. The program finds 2 paths and both have a total of 25. The countPaths function uses recursion, which calls itself to solve smaller parts of the problem.

CONCLUSION

MENDOZA	To conclude this activity, the exploration of algorithms and strategies like Recursion, Brute force, backtracking and the others shows how different strategies can be implemented and shows how efficient or how fast they can be executed. Additionally, this activity showed the difference between the Top Down Approach(Memoization) and the bottom up approach(tabulation) on how they can be used for computations, most likely, when we calculate something that has already been done or we do most of the time. Overall, this activity has shown us the correct way or how we can select the appropriate algorithmic strategies and how we can optimize our codes to get the best out of our vision.
PILAPIL	In conclusion, these programs show how recursion and dynamic programming solve problems step by step. Top-down recursion with memoization saves previous results to avoid repeating calculations, while bottom-up starts with smaller values and builds up to the answer. The path counting program uses recursion to find the number of paths with a total cost of 25. These examples show how different techniques can solve problems more efficiently.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **None**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **None**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **None**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.



Please have the honor of pledge in the comment section or at the bottom of your activity document/exam or any form of file submission.

I affirm that I will not give or receive any unauthorized help on this activity/exam and that all work will be my own.